

CSCE 608 DATABASE SYSTEMS · MODULE 05 · LECTURE NOTES

Relational Algebra and SQL

What a query means, before anyone decides how to run it.

MODULE OUTCOMES

- Express queries in relational algebra.
- Explain the gap between SQL and the algebra.
- Explain three-valued logic and the problems NULL causes.
- Trace SQL's logical evaluation order.
- Explain why declarative querying requires an optimiser.

1 Relational algebra

Operator	Notation	Meaning	SQL
Selection	$\sigma_{\text{cond}}(R)$	The rows of R satisfying a condition.	WHERE
Projection	$\pi_{\text{cols}}(R)$	The named columns, with duplicates removed — the result is a set.	SELECT DISTINCT
Union	$R \cup S$	Rows in either, duplicates removed.	UNION
Difference	$R - S$	Rows in R not in S.	EXCEPT
Cartesian product	$R \times S$	Every row of R paired with every row of S.	CROSS JOIN
Rename	$\rho_{a/b}(R)$	Rename a relation or attribute.	AS

Join is not primitive. A natural or equi-join is a selection applied to a Cartesian product: $R \bowtie S = \sigma_{\text{cond}}(R \times S)$. It is defined as a derived operator purely for convenience — and that derivation is exactly what licenses the optimiser to reorder joins and push predicates around (Module 08). An operator defined in terms of others can be re-expressed in terms of others.

CLOSURE IS THE PROPERTY EVERYTHING DEPENDS ON

Every operator takes relations and returns a relation. There is no operator that returns a scalar, or a list, or something that is not itself a valid input to every other operator.

So operators compose without restriction. The output of a selection can be projected, joined, unioned, or selected again. Subqueries, views, and common table expressions are all direct consequences — a view is simply a named expression.

And a query is therefore a tree of operators, which is precisely the structure an optimiser rewrites and an executor walks (Modules 07, 08).

Without closure there would be no query plans and no algebraic optimisation. The entire architecture of a query processor rests on this one property, which is why it is worth naming explicitly rather than treating as obvious.

SQL differs from the algebra	Consequence
Bags, not sets.	SQL keeps duplicates unless DISTINCT is requested. This is a deliberate performance decision — eliminating duplicates requires a sort or a hash — and it means SQL's UNION ALL is cheaper than UNION, and that some algebraic identities do not hold.
NULL exists.	The algebra has no such value. SQL's NULL brings three-valued logic with it, and a collection of special cases — §2.
Order exists.	ORDER BY, LIMIT, and window functions all depend on row order, which relations do not have. This is why ORDER BY is only meaningful at the outermost level, and why an ORDER BY inside a subquery or view is not guaranteed to survive.

2 NULL and three-valued logic

Codd introduced NULL to represent missing or inapplicable information, and later proposed distinguishing the two cases with separate markers. SQL adopted one NULL, and the resulting semantics are the most consistently criticised part of the language.

Expression	Evaluates to	Note
NULL = NULL	UNKNOWN	Not TRUE. Two unknown values are not known to be equal.
NULL <> NULL	UNKNOWN	Not TRUE either. They are not known to be different.
TRUE OR NULL	TRUE	The truth of the left operand settles it.
FALSE AND NULL	FALSE	Likewise.
TRUE AND NULL	UNKNOWN	Undetermined.
A WHERE clause	Passes only TRUE	UNKNOWN excludes the row, exactly as FALSE does. This is the rule that explains almost every NULL surprise.

THE NOT IN TRAP

WHERE x NOT IN (SELECT y FROM t) **returns no rows at all** if any y in the subquery is NULL.

The expansion shows why. x NOT IN (1, 2, NULL) means $x \neq 1$ AND $x \neq 2$ AND $x \neq \text{NULL}$. The final conjunct is UNKNOWN for every x , so the conjunction is either FALSE (if x equals 1 or 2) or UNKNOWN (otherwise). **It is never TRUE**, so no row ever passes the WHERE clause.

The query silently returns nothing. No error, no warning, and nothing in the text suggests a problem. It worked correctly yesterday, and then a single NULL appeared in the referenced column.

Use NOT EXISTS instead, which uses existential semantics rather than a comparison chain and behaves as expected. It is also usually faster, because it can short-circuit. **This is the most expensive NULL behaviour in practice** and it catches experienced people, because the failure is silent and data-dependent.

- **Aggregates skip NULLs.** AVG(x) divides by the number of non-NULL values, not by the row count. This is usually what you want and is worth knowing explicitly.
- **COUNT(*) counts rows; COUNT(x) counts non-NULL x .** Two genuinely different questions, distinguished by one character.
- **GROUP BY treats NULLs as equal** and collects them into a single group — even though $\text{NULL} = \text{NULL}$ is UNKNOWN. This is a direct inconsistency, and it is the clearest evidence that NULL semantics were assembled piecemeal rather than designed.
- **UNIQUE constraints usually permit multiple NULLs**, on the grounds that two NULLs are not known to be equal. The SQL standard is ambiguous and systems differ.
- **ORDER BY must place NULLs somewhere.** The standard permits first or last; PostgreSQL defaults to last for ascending, Oracle to last, MySQL to first. Specify NULLS FIRST or NULLS LAST if it matters.

3 Evaluation order

WRITTEN	LOGICALLY EVALUATED
SELECT ...	1. FROM / JOIN build the row source
FROM ...	2. WHERE filter individual rows
WHERE ...	3. GROUP BY form groups
GROUP BY ...	4. HAVING filter GROUPS
HAVING ...	5. SELECT compute output expressions
ORDER BY ...	6. DISTINCT
LIMIT ...	7. ORDER BY sort the result
	8. LIMIT

Consequence	Why
WHERE cannot reference a SELECT alias.	WHERE runs at step 2; the alias is not computed until step 5. (Some systems permit it as an extension; the standard does not.)
ORDER BY can reference a SELECT alias.	ORDER BY runs at step 7, after SELECT has computed it.
HAVING and WHERE are not interchangeable.	WHERE filters rows before grouping; HAVING filters groups after. Moving a condition from HAVING to WHERE changes the result whenever it affects which rows form the groups — and moving a non-aggregate condition the other way is a common performance mistake, since filtering before grouping is cheaper.
An aggregate cannot appear in WHERE.	Aggregates are computed during grouping at step 3. At step 2 they do not exist.

This is the *logical* order, not the *execution* order. The optimiser is free to execute in any order that produces the same result — and routinely does, pushing filters below joins and evaluating aggregates during a scan. The logical order defines the *meaning*; Module 08 covers what is done with it.

DECLARATIVE QUERYING IS THE POINT, AND THE OPTIMISER IS THE COST

A SQL query specifies *which* rows you want. It says nothing about which index to use, which join algorithm to apply, or in what order to join four tables.

So something has to decide, and that something is the optimiser (Module 08). This is Module 01's data independence, made concrete: the query is insulated from the physical structures beneath it.

The benefit is substantial. Adding an index makes existing queries faster with no change to any of them. Updated statistics change the plan. A new release's better join algorithm improves queries written a decade earlier.

The cost is loss of control. You cannot directly specify how a query runs, and when the optimiser chooses badly — which it does, for reasons Module 08 makes precise — the query is slow for reasons the text does not reveal. **This is the most common practical frustration with SQL**, and it is the direct price of the abstraction that makes everything else work.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
CMU 15-445 — Lecture 2, Modern SQL	SQL beyond the basics: window functions, CTEs, lateral joins, and the evaluation order of §3.
Date — writings on NULL and three-valued logic	The case against NULL, argued at length by someone who worked with Codd. Opinionated, and the technical content in §2 is correct.
Markus Winand — Modern SQL (free)	Excellent free material on what SQL has gained since 1992, and on the evaluation order.
PostgreSQL documentation — Queries chapter	Precise on evaluation order, NULL handling, and where the implementation differs from the standard.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- Express five queries of your own in relational algebra, then in SQL, and confirm they agree on real data.
- Demonstrate that join is derived: write an equi-join as a selection over a cross join and confirm identical results. Then compare the query plans.
- Construct a case where UNION and UNION ALL differ, and measure the cost difference.
- Build the full three-valued truth tables for AND, OR, and NOT by querying the database rather than by reading them.
- Reproduce the NOT IN trap:** write a query that returns rows, then insert one NULL and show it returns none. Rewrite with NOT EXISTS.
- Demonstrate the GROUP BY inconsistency: show that NULLs group together while NULL = NULL is UNKNOWN.
- Compare COUNT(*) and COUNT(column) on a table with NULLs and explain the difference to someone who has not seen it.
- Write a query where moving a condition from HAVING to WHERE changes the result, and one where it does not but changes the cost.
- Test NULL sort order in two different database systems and report the difference.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the six primitive relational operators and say why join is not one of them.
- 2 What is closure, and what would be impossible without it?
- 3 Give three ways SQL differs from the relational algebra.
- 4 What does `NULL = NULL` evaluate to, and what does a `WHERE` clause do with it?
- 5 Explain the `NOT IN` trap precisely and give the fix.
- 6 Give four other `NULL` inconsistencies.
- 7 State SQL's logical evaluation order and derive three of its consequences.
- 8 Why can `WHERE` not reference a `SELECT` alias when `ORDER BY` can?
- 9 What does declarative querying buy, and what does it cost?